

Università degli Studi di Padova

Dipartimento di Matematica
Laurea Triennale in Informatica
Anno Accademico 2024/2025

Progetto di Basi di Dati

RedGaming E-commerce

Alessandro Frison, Riccardo Graziani

Indice

1	Abstract	2
2	Analisi dei requisiti	2
2.1	Schema E-R	4
3	Progettazione concettuale	5
4	Progettazione logica	6
4.1	Analisi delle ridondanze	6
4.2	Eliminazione delle gerarchie	8
4.2.1	Schema E-R ristrutturato	8
4.2.2	Storage	8
4.2.3	Prodotto	8
4.2.4	Utente	10
4.3	Partizionamento e accorpamento di entità e relazioni	10
4.3.1	Indirizzo	10
4.3.2	Data	10
4.4	Schema relazionale	10
5	Implementazione PostgreSQL e Query	11
5.1	Definizioni delle query	11
5.2	Definizione degli indici	14
6	Codice C	14

1 Abstract

RedGaming è un e-commerce specializzato nella vendita di componentistica hardware (CPU, GPU, RAM, HDD, SSD, Alimentatori) per computer da gaming. In un mercato sempre in crescita come quello del gaming, RedGaming ha deciso di scalare la dimensione della propria attività per stare al passo di tale crescita: si è deciso dunque di sviluppare una nuova base di dati che potesse far fronte all'aumento di acquisti sul sito. La base di dati proposta permette a RedGaming di gestire i propri utenti: un utente può essere un cliente interessato all'acquisto dei più recenti componenti o un venditore che vuole dare più visibilità ai propri prodotti. I clienti hanno la possibilità di recensire i prodotti presenti sul sito, in modo da aiutare altri acquirenti a valutare i prodotti con il miglior rapporto qualità prezzo.

All'atto di acquisto verrà piazzato un ordine presso RedGaming, e verrà contestualmente emesso un documento di fattura e il tracciamento della spedizione, che sarà gestito da uno dei corrieri che collaborano con RedGaming. Ogni prodotto su RedGaming ha associato una certa quantità di punti: ogni ordine, quindi, fa guadagnare punti al cliente, che può spenderli per ottenere sconti sui propri acquisti. Ogni punto conferisce uno sconto di 0,10€ al momento del checkout del proprio ordine. Il valore in punti assegnato ad ogni prodotto è fissato a 1 punto ogni 10€ di prezzo del prodotto.

2 Analisi dei requisiti

Questa sezione riassume i requisiti a cui deve sottostare la base di dati.

Per poter acquistare prodotti sul sito, un **utente** deve innanzitutto registrarsi, fornendo le seguenti informazioni per poter creare il proprio account:

- **Username** univoco che lo identifichi.
- Una **password** per l'accesso al sito.
- Un indirizzo **email**.
- Un indirizzo di **residenza**, composto dai seguenti campi: via, civico, città, provincia, codice postale, nazione.

Gli utenti si distinguono in **clienti** e **venditori**, in particolare:

- I clienti possono controllare in ogni momento la quantità di **punti** accumulati ed utilizzabili per i prossimi ordini.
- Ai venditori è richiesto di inserire la propria **partita IVA**.

Ogni **prodotto** all'interno della base di dati è rappresentato dai seguenti attributi:

- Un **codice** identificativo univoco.
- Il suo **prezzo** unitario.
- Il nome del **produttore** del prodotto.
- Il **modello** del prodotto.
- Quanti **punti** l'acquisto di tale prodotto genera per l'utente.

Ogni **componente** ha inoltre delle caratteristiche specifiche:

- Una CPU possiede informazioni sul numero di **core** e sulla loro **frequenza** espressa in GHz.
- Una GPU possiede informazioni sulla quantità di **VRAM**.
- Un alimentatore possiede informazioni sul suo **wattaggio** e sulla **certificazione** 80-Plus che possiede.
- Un banco di RAM possiede informazioni sul **tipo** di RAM e sulla **dimensione** in GB.
- Lo storage possiede informazioni sulle sue **dimensioni**, la velocità di **lettura** e di **scrittura**.

Lo **storage** si divide ulteriormente in SSD e HDD:

- Un SSD possiede informazioni su tipo di **memoria NAND** usata.
- Un HDD possiede informazioni sulla velocità di **rotazione** in RPM.

I clienti hanno la possibilità di lasciare una **recensione** ai prodotti che hanno acquistato. Ogni recensione si compone dei seguenti attributi:

- Un **codice** identificativo univoco.
- Il **corpo** della recensione.
- La **data** di pubblicazione della recensione.
- La **valutazione** scelta dall'utente.

La conferma di acquisto da parte del cliente genera un **ordine**, ogni ordine è contraddistinto dai seguenti campi:

- Un **codice** identificativo univoco.
- Lo **stato** dell'ordine (In carico, spedito, annullato).
- Il numero di **punti guadagnati** dal cliente.
- Il numero di **punti spesi** dal cliente per avere uno sconto.

Una **fattura** viene emessa ogni qual volta viene generato un ordine. Ogni fattura è contraddistinta dai seguenti campi:

- Il **codice** dell'ordine a cui la fattura fa riferimento.
- La **data** di emissione della fattura.
- Il **metodo** di pagamento utilizzato.
- Il **totale** speso.
- Lo **sconto** espresso in euro.

Assieme ad ogni ordine viene emesso un tracking della relativa **spedizione**. Ogni spedizione si compone dei seguenti campi:

- Un **codice** identificativo univoco.

- La **data** di consegna prevista per tale spedizione.
- L'**indirizzo** a cui la spedizione fa riferimento, composto anche in questo caso dai campi: via, città, provincia, codice postale, civico, nazione.
- Lo **stato** corrente della spedizione.

Ogni spedizione viene presa in carico da un **corriere**, il quale si compone dei seguenti attributi:

- Il **nome** dell'azienda di spedizione, univoco.
- Il **numero** del supporto clienti.
- L'indirizzo di **mail** del supporto clienti.
- Il corpo del **tracking** della spedizione (nella forma <https://tracking.nome.com/?id=>).

2.1 Schema E-R

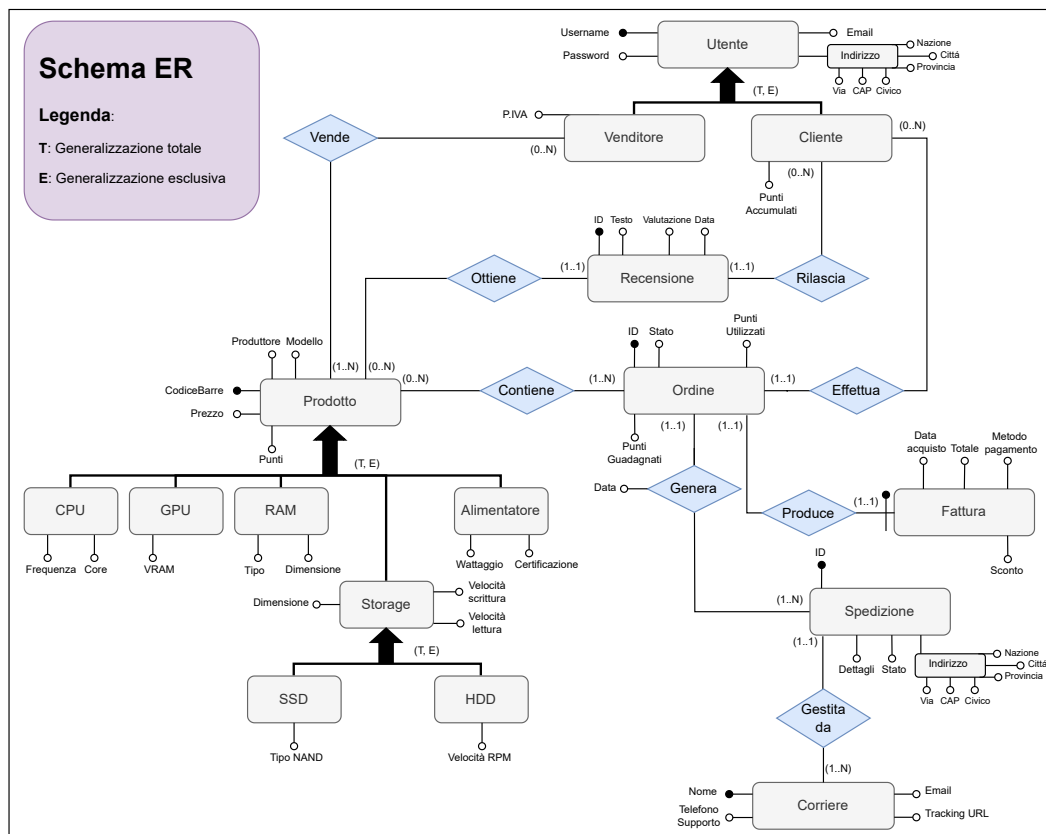


Figura 1: *Diagramma Entità-Relazione del database per la gestione dell'E-Commerce*

3 Progettazione concettuale

Dalla Figura 1, che mostra lo *Schema E-R* risultato dall'analisi dei requisiti, si ricavano le seguenti tabelle che elencano le **entità** e le **relazioni** nel database.

Entità dello schema E-R:

- **Utente**(Username, Password, Email, Indirizzo(Via, CAP, Civico, Nazione, Città, Provincia)); rappresenta un utente del sito.
- **Cliente**(PuntiAccumulati); rappresenta un utente che effettua acquisti.
- **Venditore**(PIVA); rappresenta un utente che vende prodotti.
- **Recensione**(ID, Testo, Valutazione, Data); rappresenta una recensione di un prodotto.
- **Ordine**(ID, Stato, PuntiUtilizzati, PuntiGuadagnati); rappresenta un ordine effettuato sul sito.
- **Fattura**(DataAcquisto, Totale, MetodoPagamento, Sconto); rappresenta il documento fiscale emesso a seguito di un ordine.
- **Spedizione**(ID, Dettagli, Stato, Indirizzo(Via, CAP, Civico, Nazione, Città, Provincia)); rappresenta la spedizione di uno (o più) ordini.
- **Corriere**(Nome, TelefonoSupporto, Email, TrackingURL); rappresenta il vettore che si occupa della spedizione.
- **Prodotto**(CodiceBarre, Prezzo, Produttore, Modello, Punti); rappresenta un prodotto in vendita sul sito.
- **CPU**(Frequenza, Core); rappresenta un processore in vendita.
- **GPU**(VRAM); rappresenta una scheda video in vendita.
- **RAM**(Tipo, Dimensione); rappresenta un banco di memoria in vendita.
- **Storage**(Dimensione, VelocitàLettura, VelocitàScrittura); rappresenta una memoria di massa in vendita.
- **HDD**(RPM); rappresenta un hard disk in vendita.
- **SSD**(TipoNAND); rappresenta un solid state drive in vendita.
- **Alimentatore**(Wattaggio, Certificazione); rappresenta un alimentatore in vendita.

Relazioni dello schema E-R:

- **Vende**(Venditore, Prodotto): un prodotto viene venduto da 1:N venditori, e un venditore vende 0:N prodotti.
- **Rilascia**(Cliente, Recensione): un cliente rilascia 0:N recensioni, una recensione è rilasciata da 1:1 utente.
- **Ottiene**(Prodotto, Recensione): un prodotto riceve 0:N recensioni, una recensione si riferisce ad 1:1 prodotto.

- **Effettua**(Cliente, Prodotto): un cliente effettua 0:N ordini, un ordine viene effettuato da 1:1 cliente.
- **Contiene**(Ordine, Prodotto): un prodotto è presente in 0:N ordini, un ordine si compone di 1:N prodotti.
- **Produce**(Ordine, Fattura): un ordine genera 1:1 fattura, una fattura viene emessa da 1:1 ordine.
- **Genera**(Ordine, Spedizione): un ordine viene spedito 1:1 volta, una spedizione spedisce 1:N ordini.
- **Gestita da**(Spedizione, Corriere): un corriere gestisce da 1:N spedizioni, una spedizione è gestita da 1:1 corriere.

4 Progettazione logica

4.1 Analisi delle ridondanze

L'attributo **punti** dell'entità **Prodotto** è ridondante: si può infatti calcolare dall'attributo *prezzo* dividendolo per 10, arrotondando eventuali decimali. Questo attributo viene modificato o visualizzato durante le seguenti operazioni:

- **Operazione 1 (50 al giorno)**: modifica del prezzo di un prodotto.
- **Operazione 2 (5.000 al giorno)**: visualizzazione di un prodotto.

Assumendo i seguenti volumi nella base di dati:

Concetto	Costrutto	Volume
Prodotto	E	5.000

la seguente analisi serve a determinare l'utilità o meno dell'attributo *punti*.

Mantenendo la ridondanza il costo totale delle due operazioni, assumendo costo *doppio* per scritture, ammonta a: $50 \cdot 2 \cdot 2 + 5.000 \cdot 1 = 5200$ accessi.

Concetto	Costrutto	Accessi	Tipo	Volume
Prodotto	E	1	Scrittura	x 50
Prodotto	E	1	Scrittura	x 50

Tabella 1: Operazione 1 - Scrittura di *prezzo* e *punti* con ridondanza

Concetto	Costrutto	Accessi	Tipo	Volume
Prodotto	E	1	Lettura	x 5.000

Tabella 2: Operazione 2 - Lettura per visualizzazione prodotto con ridondanza

Eliminando la ridondanza, invece, il costo totale delle due operazioni, sempre assumendo costo doppio per scritture, ammonta a: $50 \cdot 2 + 5.000 \cdot 1 = 5100$ accessi.

Data la semplicità di calcolo dell'attributo *punti*, e il piccolo risparmio in termini di accessi, si è deciso di **rimuovere** l'attributo **punti**: in tal modo si riduce anche il rischio di inconsistenza tra

Concetto	Costrutto	Accessi	Tipo	Volume
Prodotto	E	1	Scrittura	x 50

Tabella 3: Operazione 1 - Scrittura di *prezzo* senza ridondanza

Concetto	Costrutto	Accessi	Tipo	Volume
Prodotto	E	1	Lettura	x 5.000

Tabella 4: Operazione 2 - Lettura per visualizzazione prodotto senza ridondanza

il prezzo di un prodotto e il suo valore in punti, nel caso in cui ad una modifica del campo prezzo non corrisponda una modifica del campo punti.

L'attributo **Punti Accumulati** dell'entità **Cliente** è ridondante: esso è calcolabile tramite la differenza tra la somma di *Punti Guadagnati* e *Punti Utilizzati* dell'entità **Ordine**, filtrando gli ordini effettuati dal cliente. Questo attributo viene modificato o visualizzato durante le seguenti operazioni:

- **Operazione 1 (25 al giorno):** visualizzazione profilo cliente.
- **Operazione 2 (100 al giorno):** generazione ordini.

Assumendo i seguenti volumi nella base di dati:

Concetto	Costrutto	Volume
Cliente	E	10.000
Ordine	E	200.000
Effettua	R	200.000

la seguente analisi serve a determinare l'utilità o meno dell'attributo *Punti Accumulati*.

Mantenendo la ridondanza il costo totale delle due operazioni ammonta a: $100 \cdot 3 \cdot 2 + 100 + 25 = 725$ accessi al campo.

Concetto	Costrutto	Accessi	Tipo	Volume
Cliente	E	1	Lettura	x 25

Tabella 5: Operazione 1 - Visualizzazione profilo cliente con ridondanza

Concetto	Costrutto	Accessi	Tipo	Volume
Cliente	E	1	Scrittura	x 100
Ordine	E	1	Lettura	x 100
Ordine	E	1	Scrittura	x 100
Effettua	R	1	Scrittura	x 100

Tabella 6: Operazione 2 - Conferma di una ordine con ridondanza

Eliminando la ridondanza, invece, il costo totale delle due operazioni ammonta a: $21 \cdot 100 + 20 \cdot 100 + 100 \cdot 2 \cdot 2 + 25 \cdot 20 \cdot 2 + 25 = 5025$ accessi. Gli accessi nell'operazione 2 ammontano al calcolo, necessario, dei punti totali del cliente (basandosi su una media di 20 ordini salvati per cliente) e alla scrittura del nuovo Ordine su ordine ed Effettua.

Concetto	Costrutto	Accessi	Tipo	Volume
Cliente	E	1	Lettura	x 25
Ordine	E	20	Lettura	x 25
Effettua	R	20	Lettura	x 25

Tabella 7: Operazione 1 - Visualizzazione profilo cliente senza ridondanza

Concetto	Costrutto	Accessi	Tipo	Volume
Ordine	E	21	Lettura	x 100
Effettua	R	20	Lettura	x 100
Ordine	E	1	Scrittura	x 100
Effettua	R	1	Scrittura	x 100

Tabella 8: Operazione 2 - Conferma di un ordine senza ridondanza

Dall'analisi risulta dunque conveniente **mantenere** l'attributo ridondante *Punti Accumulati* al fine di ottimizzare gli accessi durante le operazioni di routine.

4.2 Eliminazione delle gerarchie

Nello schema concettuale originario (Sezione 3), tutte le generalizzazioni adottate sono di tipo **esclusivo e totale**: ogni entità figlia specializza un'unica super-entità senza sovrapposizioni, e tutte le istanze della super-entità devono appartenere necessariamente a una delle sottoclassi definite. Per ottimizzare il modello logico-relazionale si procede all'eliminazione di queste gerarchie attraverso una riconfigurazione mirata.

4.2.1 Schema E-R ristrutturato

Nelle successive sezioni vengono indicate le procedure che spiegano le modifiche che hanno portato allo schema ristrutturato di Figura 2.

4.2.2 Storage

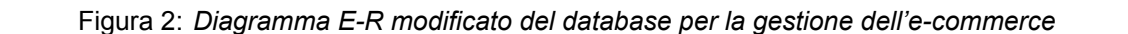
La generalizzazione totale di **Storage** è stata eliminata facendo discendere le proprietà comuni nei due figli **HDD** e **SSD**. Questa scelta implementativa semplifica la successiva ristrutturazione della gerarchia di Prodotto. In particolare è stato scelto questo approccio per i seguenti motivi:

- Semplicità di scomposizione della gerarchia di Prodotto
- Logicamente coerente, infatti *HDD* e *SSD* rappresentano due prodotti separati e diversi
- In questo modo si evita di creare inutilmente tabelle contenenti pochi attributi.

4.2.3 Prodotto

Il modello concettuale iniziale prevedeva un'entità generica **Prodotto** con attributi comuni a tutti i componenti hardware. Per ottimizzare la struttura del database ed evitare ridondanze, è stata implementata una specializzazione mediante relazioni 1:1 a seguito della procedura di normalizzazione della gerarchia di Storage:

- *IS-CPU* per **CPU** (attributi: core, frequenza)



- Questa soluzione permette innanzitutto di limitare le relazioni tra i vari prodotti e le entità come *Utente*, *Recensione* e *Ordine*: mantenere l'entità Prodotto consente di avere una struttura più semplice e capace di essere ampliata in futuro con nuovi prodotti. Inoltre, ogni entità specializzata mantiene solo gli attributi rilevanti.

- Ottimizzazione delle operazioni transazionali.
- Gestione semplificata degli attributi condivisi (modello, prezzo, produttore).

4.2.4 Utente

La gerarchia completa dell'entità **Utente** è stata ristrutturata attraverso l'accorpamento delle entità **Cliente** e **Venditore** nell'entità padre *Utente*, considerando che le entità figlie possiedono pochi attributi caratterizzanti. Questa soluzione offre i seguenti vantaggi:

- **Riduzione della complessità dello schema:** L'eliminazione delle sottoclassi riduce il numero di tabelle e relazioni, semplificando le operazioni di query e mantenimento.
- **Flessibilità:** L'uso di un attributo discriminante **tipo** permette di distinguere tra clienti e venditori senza necessità di tabelle separate.

4.3 Partizionamento e accorpamento di entità e relazioni

4.3.1 Indirizzo

Per evitare ridondanze dell'attributo composto **indirizzo**, si è introdotta un'entità **Indirizzo** condivisa tra **Utente** e **Spedizione**. Come chiave primaria si è scelta la combinazione *CAP + Via + Civico*, ritenuta sufficientemente univoca nel dominio applicativo. Questa soluzione permette di: normalizzare la struttura dati, ridurre gli attributi ripetuti e mantenere la flessibilità per eventuali estensioni future, semplificando al contempo le tabelle correlate.

4.3.2 Data

Nel modello E-R, l'attributo **Data** associato alla relazione **Genera** è stato mantenuto nella tabella dell'entità **Ordine**. Questa soluzione risulta ottimale in quanto l'attributo è logicamente strettamente legato all'entità *Ordine*.

4.4 Schema relazionale

Lo schema ristrutturato presente nella sezione 4.2.1 rappresenta lo **schema logico** in forma visiva. Di seguito vengono elencate tutte le tabelle rappresentabili da tale schema logico:

- **Utente**(Username, Password, Email, Tipo, PIVA*, PuntiAccumulati*)
- **Indirizzo**(Via, CAP, Civico, Città, Provincia, Nazione, Utente)
 - Indirizzo.Utente → Utente.Username
- **Recensione**(ID, Testo, Valutazione, Data, Cliente, Prodotto)
 - Recensione.Cliente → Utente.Username
 - Recensione.Prodotto → Prodotto.CodiceBarre
- **Ordine**(ID, Stato, Data, PuntiGuadagnati, PuntiUtilizzati, Cliente, Spedizione)
 - Ordine.Cliente → Utente.Username
 - Ordine.Spedizione → Spedizione.ID
- **Fattura**(Ordine, DataAcquisto, Totale, Sconto, MetodoPagamento)
 - Fattura.Ordine → Ordine.ID
- **Spedizione**(ID, Dettagli, Stato, Corriere, Via, CAP, Civico)
 - Spedizione.Corriere → Corriere.Nome
 - Spedizione.(Via, CAP, Civico) → Indirizzo.(Via, CAP, Civico)

- **Corriere**(Nome, TelefonoSupporto, Email, TrackingURL)
- **Prodotto**(CodiceBarre, Prezzo, Produttore, Modello)
- **CPU**(Prodotto, Frequenza, Core)
 - CPU.Prodotto → Prodotto.CodiceBarre
- **GPU**(Prodotto, VRAM)
 - GPU.Prodotto → Prodotto.CodiceBarre
- **RAM**(Prodotto, Tipo, Dimensione)
 - RAM.Prodotto → Prodotto.CodiceBarre
- **HDD**(Prodotto, Dimensione, VelocitaLettura, VelocitaScrittura, RPM)
 - HDD.Prodotto → Prodotto.CodiceBarre
- **SSD**(Prodotto, Dimensione, VelocitaLettura, VelocitaScrittura, NAND)
 - SSD.Prodotto → Prodotto.CodiceBarre
- **Alimentatore**(Prodotto, Wattaggio, Certificazione)
 - Alimentatore.Prodotto → Prodotto.CodiceBarre
- **Vende**(Venditore, Prodotto)
 - Vende.Venditore → Utente.Username
 - Vende.Prodotto → Prodotto.CodiceBarre
- **Contiene**(Ordine, Prodotto)
 - Contiene.Ordine → Ordine.ID
 - Contiene.Prodotto → Prodotto.CodiceBarre

5 Implementazione PostgreSQL e Query

Il file *redgaming.sql* contiene il codice SQL usato per la **generazione** delle tabelle, il loro **popolamento** e il codice per la **definizione** delle query utilizzate. Infine è presente la definizione di un **indice** creato in modo da migliorare le prestazioni di una delle query.

5.1 Definizioni delle query

Query 1: Visionare l'elenco dei prodotti di un ordine effettuato da un cliente con il loro tipo e relativo prezzo.

```
SELECT u.Username AS Cliente, o.ID AS ID_Ordine, p.CodiceBarre AS Codice_Prodotto,
       p.Prezzo, p.Produttore, p.Modello,
       CASE
         WHEN cpu.Prodotto IS NOT NULL THEN 'CPU'
         WHEN gpu.Prodotto IS NOT NULL THEN 'GPU'
         WHEN ram.Prodotto IS NOT NULL THEN 'RAM'
         WHEN hdd.Prodotto IS NOT NULL THEN 'HDD'
         WHEN ssd.Prodotto IS NOT NULL THEN 'SSD'
```

```

        WHEN alimentatore.Prodotto IS NOT NULL THEN 'Alimentatore'
        ELSE 'Altro'
    END AS Tipo_Prodotto
FROM Utente u
JOIN Ordine o ON u.Username = o.Cliente
JOIN Contiene c ON o.ID = c.Ordine
JOIN Prodotto p ON c.Prodotto = p.CodiceBarre
LEFT JOIN CPU cpu ON p.CodiceBarre = cpu.Prodotto
LEFT JOIN GPU gpu ON p.CodiceBarre = gpu.Prodotto
LEFT JOIN RAM ram ON p.CodiceBarre = ram.Prodotto
LEFT JOIN HDD hdd ON p.CodiceBarre = hdd.Prodotto
LEFT JOIN SSD ssd ON p.CodiceBarre = ssd.Prodotto
LEFT JOIN Alimentatore alimentatore ON p.CodiceBarre = alimentatore.Prodotto
WHERE u.Username = 'daniele.bianchi' AND o.ID = 21;

```

	cliente character varying (50)	id_ordine integer	codice_prodotto character varying (20)	prezzo real	produttore character varying (50)	modello character varying (50)	tipo_prodotto text
1	daniele.bianchi	21	60000000000010	149.99	Corsair	HX1000 1000W 80+ Platinum	Alimentatore
2	daniele.bianchi	21	40000000000007	129.99	Seagate	IronWolf 6TB NAS	HDD
3	daniele.bianchi	21	40000000000010	89.99	Western Digital	Blue 1TB 7200RPM	HDD
4	daniele.bianchi	21	72000000000003	159.99	Crucial	MX500 1TB	SSD
5	daniele.bianchi	21	72000000000018	155	Sabrent	Rocket 1TB	SSD
6	daniele.bianchi	21	50000000000003	89.99	Western Digital	Blue SN550 500GB	SSD
7	daniele.bianchi	21	50000000000013	89.99	Kingston	KC600 512GB	SSD
8	daniele.bianchi	21	40000000000014	119.99	Toshiba	X300 6TB 7200RPM	HDD
9	daniele.bianchi	21	60000000000005	59.99	Thermaltake	Smart 600W 80+ Bronze	Alimentatore

Query 2: Visionare gli N (es. 5) utenti che hanno speso di più, in media, ordinati per spesa.

```

SELECT u.Username AS Cliente, ROUND(AVG(f.Totale), 2) AS MediaSpesa
FROM Utente u
JOIN Ordine o ON u.Username = o.Cliente
JOIN Fattura f ON o.ID = f.Ordine
WHERE f.DataAcquisto >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY u.Username
ORDER BY MediaSpesa DESC LIMIT 5;

```

	cliente character varying (50)	mediaspesa numeric
1	riccardo.damico	639.00
2	lucia.verdi	609.40
3	simone.lombardi	512.75
4	federico.rossi	459.00
5	giulia.verdi	429.50

Query 3: Lista dei venditori che hanno in vendita più di N (es. 10) prodotti.

```

SELECT u.Username AS Venditore, COUNT(v.Prodotto) AS NumeroProdotti
FROM Utente u

```

```

JOIN Vende v ON u.Username = v.Venditore
JOIN Prodotto p ON v.Prodotto = p.CodiceBarre
WHERE u.Tipo = 'Venditore'
GROUP BY u.Username
HAVING COUNT(v.Prodotto) >= 10
ORDER BY NumeroProdotti DESC;

```

	venditore character varying (50)	numeroprodotti bigint
1	gamerhub	14
2	techstore	12
3	elena.rossi	11
4	vittoria.rossi	11
5	andrea.rossi	11
6	paolo.ferri	11
7	filippo.ricci	10
8	andrea.martinelli	10
9	alessandra.marini	10

Query 4: Elencare, per ogni corriere, il numero di spedizioni effettuate, il numero medio di prodotti gestiti e il valore di tutte le spedizioni.

```

CREATE VIEW ProdottiPerOrdine AS
SELECT Ordine, COUNT(*) AS NumeroProdotti
FROM Contiene
GROUP BY Ordine;

SELECT c.Nome AS Corriere, COUNT(DISTINCT s.ID) AS NumeroSpedizioni,
       ROUND(AVG(p.NumeroProdotti), 2) AS MediaProdottiGestiti,
       ROUND(SUM(f.Totale), 2) AS ValoreTotaleSpedizioni
FROM Corriere c
JOIN Spedizione s ON c.Nome = s.Corriere
JOIN Ordine o ON o.Spedizione = s.ID
JOIN Fattura f ON o.ID = f.Ordine
JOIN ProdottiPerOrdine p ON o.ID = p.Ordine
GROUP BY c.Nome
ORDER BY NumeroSpedizioni DESC;

```

	corriere character varying (50)	numerospedizioni bigint	mediaprodottigestiti numeric	valoretotalespedizioni numeric
1	GLS	6	3.14	2159.82
2	SDA	3	2.67	849.78
3	TNT	3	4.00	1906.13
4	Bartolini	2	5.00	803.20
5	Poste Italiane	2	2.00	468.49
6	Amazon Logistics	1	8.00	429.50

Query 5: Top 5 utenti più attivi sul sito che hanno rilasciato recensioni, considerando la media delle recensioni rilasciate.

```
SELECT r.Cliente AS Username, COUNT(r.ID) AS NumeroRecensioni,
       ROUND(AVG(r.Valutazione), 2) AS MediaValutazione
FROM Recensione r
GROUP BY r.Cliente
ORDER BY NumeroRecensioni DESC, MediaValutazione DESC
LIMIT 5;
```

	username character varying (50)	numerorecensioni bigint	mediavalutazione numeric
1	giulia.verdi	12	3.50
2	chiara.deangelis	10	3.60
3	silvia.menegatti	8	4.25
4	francesca.rizzo	8	3.63
5	marco.bianchi	7	3.00

5.2 Definizione degli indici

Si è deciso di migliorare le performance della **Query 1** attraverso la creazione di un indice B+Tree mirato: sulla tabella *Ordine* viene creato un indice sul campo **Cliente**, il quale è chiave esterna verso la tabella *Utenti*.

```
CREATE INDEX idx_ordine_cliente ON Ordine(Cliente)
```

Entrambe le tabelle sono spesso utilizzate durante operazioni di routine all'interno dell'e-commerce. Ad esempio, se un utente accede alla sua area personale, potrebbe voler visualizzare lo storico dei suoi ordini: l'indice appena creato permette di velocizzare tale operazione, anche considerando il numero crescente di utenti e relativi ordini.

6 Codice C

Il file *redgaming.c* contiene il codice necessario ad accedere alla base dati, il quale può essere compilato ed eseguito tramite i comandi:

```
gcc redgaming.c -o redgaming -I dependencies/include/ -lpq
./redgaming
```

Verrà mostrato all'utente un menu con le seguenti opzioni:

```
[-1 ] >> Esci
[ 0 ] >> Query disponibili
[1-5] >> Seleziona query
```

tramite cui l'utente potrà scegliere se chiudere l'applicazione, visualizzare, in formato testuale, le query disponibili o scegliere di eseguire una delle cinque query.

In particolare, le query 1 – 3 sono state **parametrizzate** in modo da dare la libertà all'utente di ricercare i dati che desidera:

- **Query 1:** è possibile specificare lo **username** e il **numero** dell'ordine che si desidera visualizzare (i valori consigliati sono daniiele.bianchi per lo *username* e 21 per il *numero* dell'ordine).
- **Query 2:** è possibile specificare il **numero** di utenti di cui si desidera conoscere la spesa media (il valore di default è impostato a 5).
- **Query 3:** è possibile specificare il **numero** minimo di oggetti che un venditore ha in vendita (il valore di default è impostato a 10).

Sono state definite alcune funzioni helper utilizzate per l'acquisizione di input utente e la stampa dei risultati delle query. In particolare, le funzioni:

```
int read_int_from_stdin(const char* message);
char* read_string_from_stdin(char* input, int size, const char* message);
```

permettono di leggere input numerici e testuali usando la funzione fgets, che permette una migliore error handling di un semplice scanf, e rimuovono eventuali caratteri di escape rimasti. Per poter stampare i risultati delle query in maniera tabulare sul terminale sono state definite le funzioni:

```
void print_separator(int* col_widths, int cols);
void print_query_result(PGresult* result);
```

La funzione principale del programma é la funzione:

```
PGresult* choose_query(PGconn* connection);
```

la quale gestisce il menu utilizzabile dall'utente, riceve eventuali parametri in ingresso e chiama le funzioni di execute appropriate, ossia:

```
PGresult* execute_query(PGconn* connection, const char* query);
PGresult* execute_parametric_query(PGconn* connection, const char* query,
                                   int num_parameters, const char* const *parameters);
```

che eseguono le query fornite ed effettuano un controllo sulla correttezza del risultato ritornato. Il main del programma, infine, stabilisce la connessione con il database chiedendo in input i vari parametri all'utente, richiama la funzione *choose_query* in un while ed infine chiude la connessione una volta che l'utente decide di chiudere il programma.